DAY 04 / 30 | MYDFIR 30-DAY MICROSOFT CHALLENGE

Microsoft Sentinel & KQL Fundamentals

Hands-on learning journal • Failed logon analysis in the SecurityEvent table • Bo Mejdahl

Introduction

I am building practical SOC analyst skills, and Day 4 of the MyDFIR 30-Day Microsoft Challenge put me inside a real Azure Log Analytics workspace with Microsoft Sentinel. Alongside the Microsoft Learn modules on Sentinel and KQL, this exercise let me query Windows security events, filter failed logons, pick out useful fields, and count and rank the results. My goal is not only to run queries, but to understand what the results mean and what an analyst should look at next.

I learn best with a picture in my head, so this journal explains every step with one running analogy: **Sentinel is a big hotel, and I am the night receptionist reading the logbook.**

The analogy for the whole day

Real thing	Hotel analogy
Log Analytics workspace	The hotel's **records room**, where every logbook is stored
`SecurityEvent` table	The **reception logbook**: every key-card swipe and door event gets a line
One row in the table	One **entry** in the logbook
Event ID 4625	**Wrong key card**: the door did not open (a failed Windows logon)
A KQL query	**Instructions** I give the receptionist about what to look up
The pipe symbol `|`	**"...and then"**: each step works on what the previous step handed over

Learning objectives

Navigate Microsoft Sentinel and Log Analytics; identify the SecurityEvent table; practise `where`, `take`, `project`, `summarize` and `sort by`; examine failed Windows logons (Event ID 4625); and document reproducible exercises for my cybersecurity portfolio.

Lab environment: Microsoft Azure • Microsoft Sentinel • Log Analytics workspace • SecurityEvent table (Windows security telemetry, training data).

How a KQL query flows

```
SecurityEvent
| summarize EventCount = count() by EventID
| sort by EventCount desc
```

HOTEL ANALOGY

"*Open the reception logbook...* **and then** *make a tally of how many times each type of event happened...* **and then** *put the biggest tally on top.*"

A high number alone does not mean something bad is happening. A busy hotel has lots of normal door swipes too. The tally tells me **where to look first**.

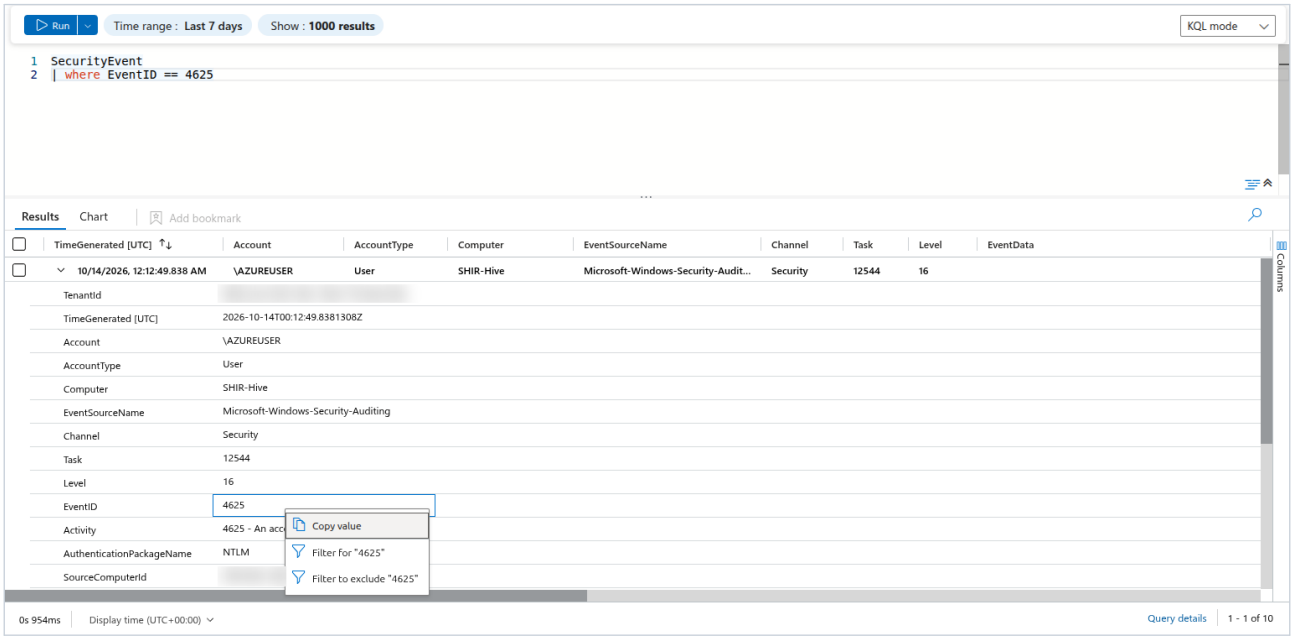
Exercise 1 — Find failed Windows logons

```
SecurityEvent
| where EventID == 4625
```

HOTEL ANALOGY

`where` **is the receptionist's highlighter.** "Only show me the lines where someone used a wrong key card."

Event ID **4625** means a failed logon. The cause can be innocent (a mistyped password, an old saved password on a service) or suspicious (someone guessing passwords). This query is the starting point, not the verdict. In the expanded row I can also see the authentication package (NTLM) and the target computer, which are useful clues for later.



My query: filtering SecurityEvent for Event ID 4625, with one event expanded. Tenant and source identifiers blurred.

Exercise 2 — Look at a sample first

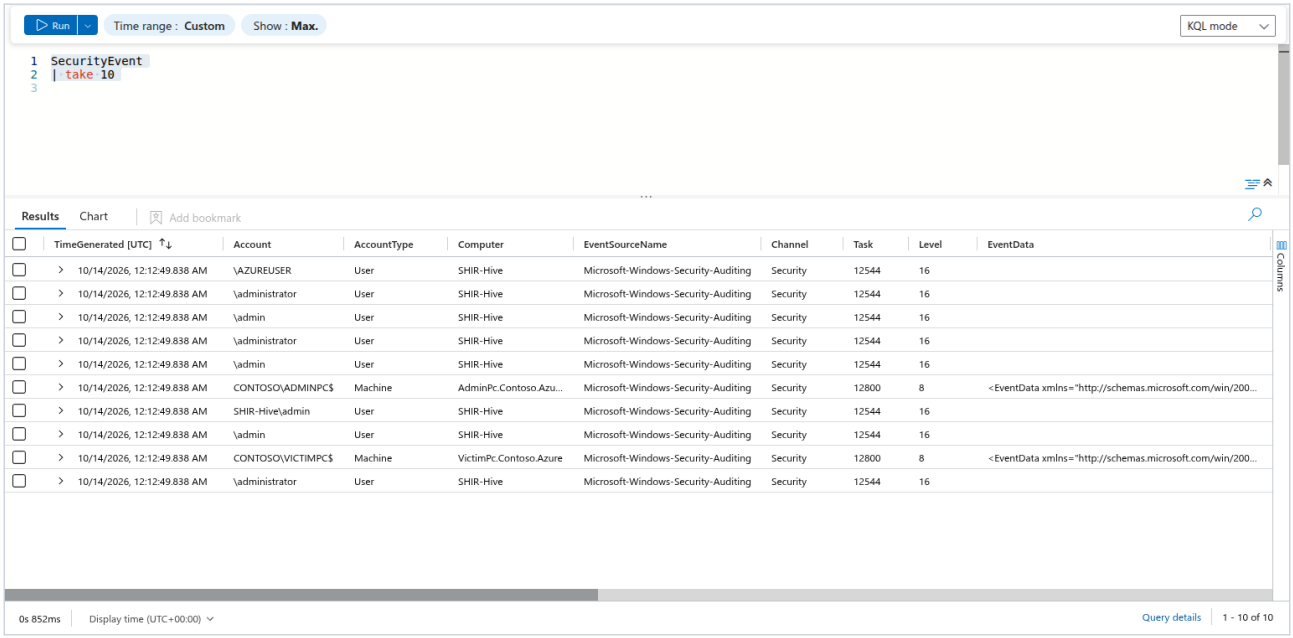
```
SecurityEvent
| take 10
```

HOTEL ANALOGY

`take` **is flipping the logbook open and reading any 10 lines.** It is a quick way to see which columns exist before writing a real query.

WATCH OUT

`take` does **not** return the newest 10 events, just *any* 10. To see the latest events, I need an explicit sort.



My query: ten sample events from the SecurityEvent table.

WHAT I NOTICED

Even in this small sample, the account names stand out: `admin`, `administrator` and AZUREUSER. These are classic guessable names, the kind an attacker tries first. In hotel terms: someone is trying the doors marked "Manager" and "Staff" rather than random guest rooms. That is a pattern worth checking.

Exercise 3 — Pick the useful columns and put them in order

```
SecurityEvent
| where EventID == 4625
| project TimeGenerated, TimeCollected, Account, Computer, EventID
| sort by TimeCollected asc
```

HOTEL ANALOGY

`project` **is photocopying only the columns I care about** (when, who, which door) instead of the whole page.
`sort by … asc` **lines the entries up from oldest to newest**, like replaying the night in order.

Lesson learned: the query runs, but `TimeCollected` turned out to hold no values in my dataset (see Exercise 8), so this sort does not really put anything in order. I always check what a column actually contains in my own workspace, not just whether it exists.

WHY MY TABLE NAMES DIFFER FROM THE VIDEO

In May 2026 Microsoft updated the Sentinel Training Lab and its sample data. That is why the course video queries a table called `SecurityEvent_CL` with a column `EventID_s`, while my workspace uses `SecurityEvent` and `EventID`. Same data idea, different labels. In hotel terms: the hotel printed new logbooks with renamed columns, so I read the headings before copying the old instructions.

Exercise 4 — Count events by type

```
SecurityEvent
| summarize EventCount = count() by EventID
```

HOTEL ANALOGY

`summarize count() by EventID` **is the receptionist's tally sheet:** one line per event type, with the number of times it happened.

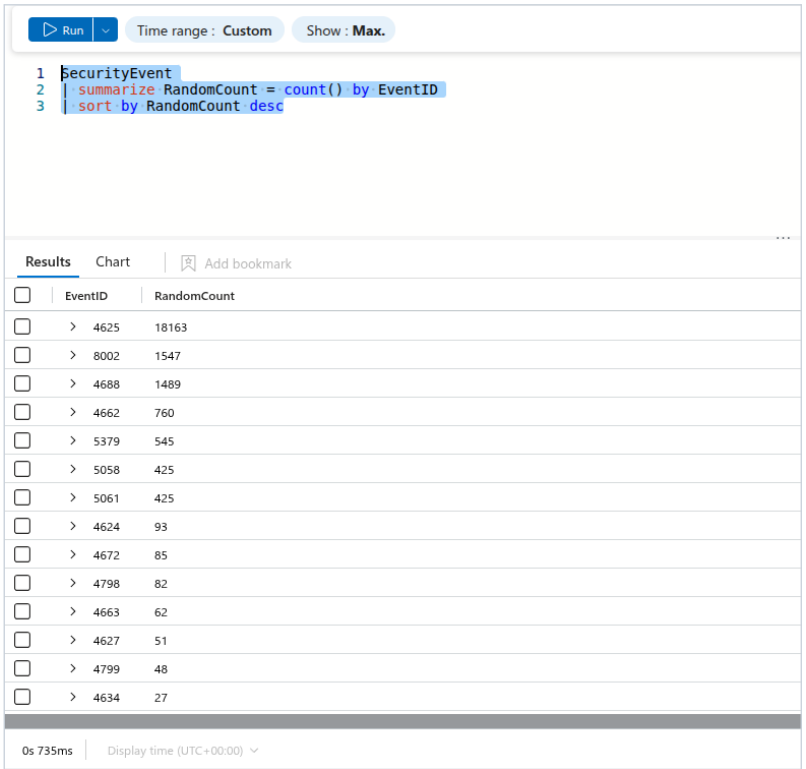
Exercise 5 — Rank event types by frequency

```
SecurityEvent
| summarize EventCount = count() by EventID
| sort by EventCount desc
```

HOTEL ANALOGY

The same tally sheet, but with the **busiest event type on top**.

I renamed the count column to `EventCount` for clarity; my screenshot shows the name from the course notes, `RandomCount`.



My query: Event IDs ranked by count (15 of 22 rows shown).

What the results told me

Failed logons (4625) dominate the dataset with 18,163 entries, more than ten times the next event type (1,547). In hotel terms: a flood of wrong key cards. Combined with the admin-style account names from Exercise 2, this is exactly the kind of pattern an analyst would investigate next. Successful logons (4624) appear only 93 times.

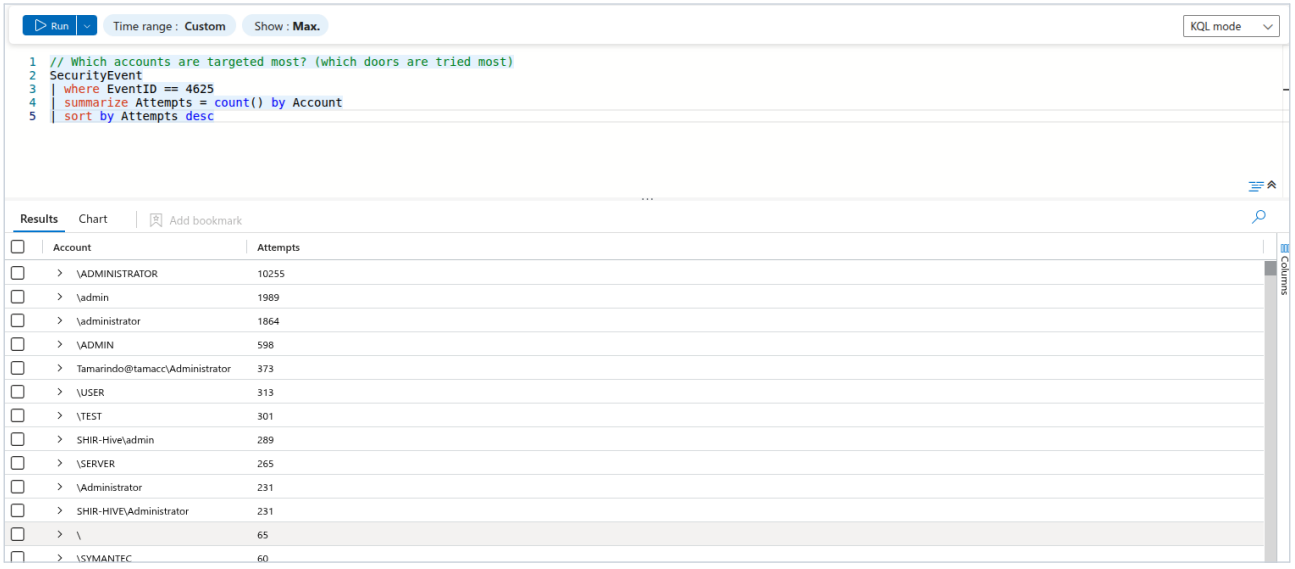
Event ID	Count	Meaning	Hotel analogy
4625	18,163	An account failed to log on	Wrong key card
4688	1,489	A new process was created	Someone switched on equipment in a room
4624	93	An account logged on successfully	Key card worked, door opened
4672	85	Special privileges assigned at logon	A **master key** was used
4798 / 4799	82 / 48	Group membership was enumerated	Someone read the **staff list**
4634	27	An account logged off	Guest left the room

Exercise 6 — Which accounts are targeted most?

```
SecurityEvent
| where EventID == 4625
| summarize Attempts = count() by Account
| sort by Attempts desc
```

HOTEL ANALOGY

Counting wrong key cards per door. Instead of "how many wrong key cards in total?", I ask the receptionist: "which doors were tried the most?"



My query: failed logons grouped by account, most targeted first (13 of the top rows shown).

Account	Attempts	What it suggests
\ADMINISTRATOR	10,255	The built-in admin account: **56.5%** of all failed logons on its own
\admin, \administrator, \ADMIN, \Administrator	1,989 / 1,864 / 598 / 231	The same targets typed in different upper/lower case
SHIR-Hive\admin, SHIR-HIVE\Administrator, Tamarindo@tamacc\Administrator	289 / 231 / 373	Admin accounts with a computer or domain name in front
\USER, \TEST, \SERVER, \SYMANTEC	313 / 301 / 265 / 60	Generic default names, typical of an automated wordlist

WHAT I FOUND

Admin-style accounts add up to **15,830 of 18,163 failed logons (about 87%)**. The rest are generic names like USER, TEST and SERVER. This pattern is consistent with an **automated brute-force attempt** using a list of common default usernames, rather than a real employee mistyping a password.
In hotel terms: someone walked the corridor trying the **Manager's office** door over 10,000 times, then rattled the doors marked "Test", "Server" and "Staff". Guests who forgot their room number do not behave like that.

WATCH OUT

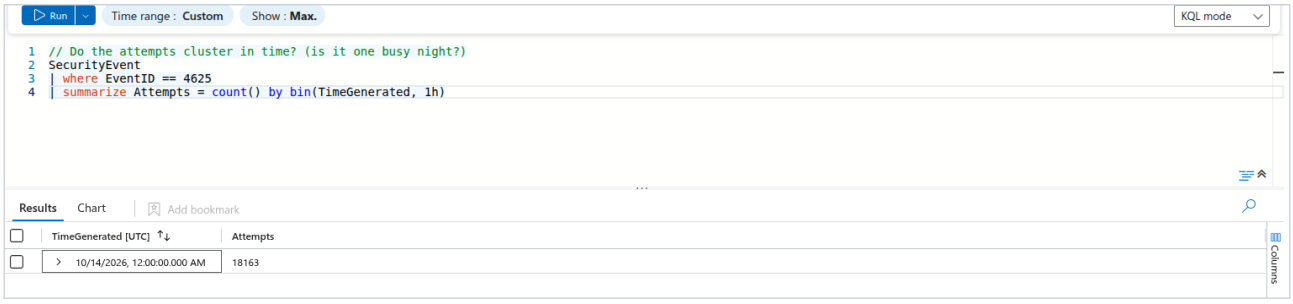
KQL groups text **case-sensitively**, so `ADMINISTRATOR`, `administrator` and `Administrator` are counted as three different accounts. To see the true total per account, normalise the names first with `tolower()`.

Exercise 7 — Do the attempts cluster in time?

```
SecurityEvent
| where EventID == 4625
| summarize Attempts = count() by bin(TimeGenerated, 1h)
```

HOTEL ANALOGY

`bin(TimeGenerated, 1h)` **sorts the logbook entries into one-hour buckets**, like asking: "how many wrong key cards between midnight and 1 am, between 1 and 2 am...?"



My query: failed logons per hour. Everything lands in a single bucket.

WHAT I FOUND

The result has only **one row: all 18,163 failed logons in the same hour** (14 Oct 2026, 00:00 UTC). At first sight that looks like one massive attack burst. But this is a lesson about the data, not the attacker: in Exercise 2, every sample event had the **exact same timestamp** (12:12:49.838 AM). The training data was ingested into my workspace **from a file, in one go**, so `TimeGenerated` shows **when the logs arrived**, not when the logons were really attempted. Exercise 8 proves it.

In hotel terms: an old logbook was photocopied into the new one at midnight. Every entry now carries the photocopy date, so it cannot tell me which night the doors were really tried.

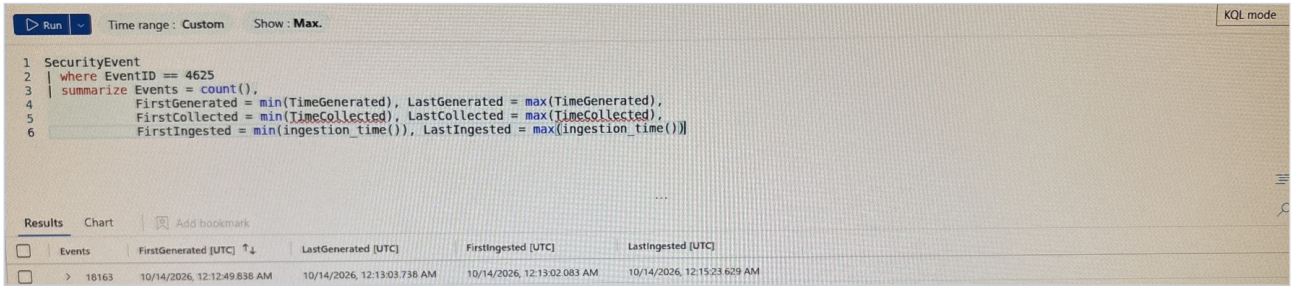
Why this matters for a SOC analyst: in a real environment, a timeline is one of the strongest pieces of evidence. Before trusting it, I need to know which time field I am reading: the time the event happened, or the time it was collected.

Exercise 8 — Which clock can I trust?

```
SecurityEvent
| where EventID == 4625
| summarize Events = count(),
    FirstGenerated = min(TimeGenerated),
    LastGenerated  = max(TimeGenerated),
    FirstCollected = min(TimeCollected),
    LastCollected  = max(TimeCollected),
    FirstIngested  = min(ingestion_time()),
    LastIngested   = max(ingestion_time())
```

HOTEL ANALOGY

Checking the photocopier's own log. Instead of trusting the time written on each logbook line, I compare three clocks: when the line was written, when the guard collected it, and when it reached the records room.



My query: first and last value of each time field for all 18,163 failed logons.

Clock	First	Last	Spread
TimeGenerated	12:12:49.838	12:13:03.738	about 14 seconds
Ingestion time	12:13:02.083	12:15:23.629	about 2½ minutes
TimeCollected	no values	no values	empty

WHAT I FOUND

All 18,163 failed logons were "generated" within **14 seconds** and reached the workspace within the next **2½ minutes**. This matches what I did: I ingested the training data from a file. `TimeGenerated` records the upload, and `TimeCollected` returned no values, so this dataset has **no trustworthy original event time**.
What I can still answer: **who** was targeted and **how often**. What I cannot answer: **when**, or **in what order**. Naming the limits of the evidence is part of the analysis.

Key takeaways

- KQL turns a giant logbook into a short, focused answer, one pipe step at a time.
- `where` filters, `take` samples, `project` picks columns, `summarize` counts, `sort` orders, `bin()` groups by time.
- About 87% of the failed logons targeted admin-style accounts: a pattern consistent with automated brute force.
- Text grouping is case-sensitive, so the same account can hide in several rows.
- Check which clock a timestamp comes from before building a timeline. Here it showed my file upload, not the attack.
- A big number is a **lead, not proof**. This is training data, not a confirmed attack.

Next steps

The most important question is still open: **did any of these attempts succeed?** In hotel terms: after all those wrong key cards, did any door actually open?

```
// Did the targeted accounts ever log on successfully?
SecurityEvent
| where EventID in (4624, 4625)
| extend AccountLower = tolower(Account)
| where AccountLower has "admin"
| summarize Failed = countif(EventID == 4625),
            Succeeded = countif(EventID == 4624) by AccountLower
| sort by Failed desc
```

After that: find where the attempts come from (source IP address or workstation name, when those fields are available). I will keep documenting each challenge day with the query, its purpose, my observations and lessons learned, building towards SOC analysis, cloud security and AI security.